



VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Chapter 6: FUNCTIONS

September 2025

Content

1 Function declarations and definitions

2 Pass-by-value

3 Variable scope

4 Variable storage classes

5 Pass-by-reference

6 Passing arrays to functions

Content

1 Function declarations and definitions

2 Pass-by-value

3 Variable scope

4 Variable storage classes

5 Pass-by-reference

6 Passing arrays to functions

Function declarations

- User-defined program units are called *subprograms*. In C++ all subprograms are referred to as **functions**.
- Defining a Function: The lines that compose a function within a C++ program are called a **function definition**.
- The syntax for declaring & defining a function:

```
<data_type>  <name_of_function> ( <parameters> ) {  
    // many statements/variable/constant declarations  
    <function body>  
}
```

- To call a function

```
<data_type> result = <name_of_function> (<param1, param2, ...>)
```

Function definitions

- A function definition consists of four parts:
 - A reserved word indicating the data type of the function's return value.
 - The function name
 - Any parameters required by the function, contained within (and).
 - The function's statements/declarations enclosed in curly braces { }.
- Example 6.1.1:

```
void FindMax (int x, int y) {  
    int maxnum;  
    maxnum = (x >= y) ? x : y;  
    cout << "\n The maximum of the 2 numbers is "  
        << maxnum << endl;  
    return;  
}
```

How to call a function

- You designate a **data type** for function since it will **return a value** from a function **after it executes**.
- Variable names that will be used in the function header line are called **formal parameters**.
- To execute a function, you must invoke, or call, it by a function call.
- The values or variables that you place within the parentheses of a function call statement are called **actual parameters**.
- Example:

```
int firstNum = 5, secNum = 8;  
findMax (firstNum, secNum);
```

Function Prototypes

- A **function prototype** declares to the compiler that you intend to use a function later in the program.
- If you try to call a function at any point in the program **prior** to **its function prototype or function definition**, you will receive an **error** at compile time.

Example 7.3.2: Finding the maximum of three integers

```
#include <iostream>
using namespace std;

// function prototype (forward declaration)
int maximum (int, int, int);

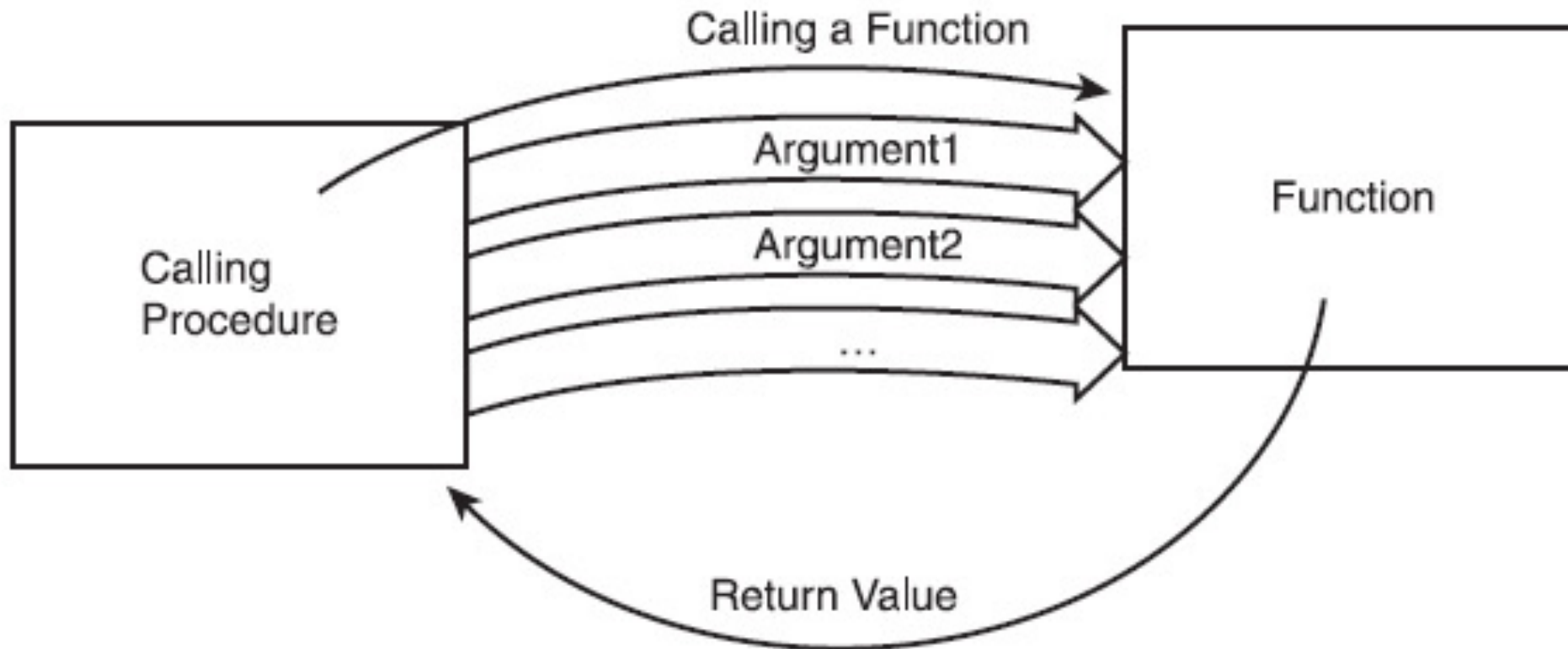
int main()
{
    int a, b, c;
    cout << "Enter three integers: ";
    cin >> a >> b >> c;
    cout << "Maximum is: " << maximum (a, b, c) << endl;
    return 0;
}
```

Example 7.3.2: Finding the maximum of three integers (cond.)

```
// Function maximum definition
// x, y and z are parameters to the maximum function definition
int maximum (int x, int y, int z)
{
    int max = x;
    if ( y > max )
        max = y;
    if ( z > max )
        max = z;
    return max;
}
```

- The output of the above program:
Enter three integers: 22 85 17
Maximum is: 85

Conceptual diagram of the process of calling a function



Content

1 Function declarations and definitions

2 Pass-by-value

3 Variable scope

4 Variable storage classes

5 Pass-by-reference

6 Passing arrays to functions

Pass by Value

- If a variable is one of the **actual parameters** in a **function call**, the called function receives a **copy** of the values stored in the variable.
- After the values are **passed** to the called function, control is transferred to the called function.
- Example: The statement `findMax (firstnum, secnum);`
 - calls the function `findMax`
 - causes the values currently residing in the variables `firstnum` and `secnum` to be passed to `findMax` function.
- The method of passing values to a called function is called **pass by value**.

Returning values

- To actually **return** a value from a function, you must include the **return statement** within the called function.
- The syntax for the return statement is either
return expression;
or
return ;
- Values **passes back and forth** between functions must be of **the same data type**.

Content

1 Function declarations and definitions

2 Pass-by-value

3 Variable scope

4 Variable storage classes

5 Pass-by-reference

6 Passing arrays to functions

Variable scope

- **Scope** refers to **where** in your program a declared variable or constant is **allowed to be used**.
- A variable/constant declaration with **Global scope** is one that is declared **outside** of any functions or classes and is **accessible** throughout the **entire program**.
- A variable/constant declaration with **Local scope** is one that is declared **inside** a function and is **accessible** only **within** that **specific function**.

Example 6.3.1

```
#include <iostream>
using namespace std;
int x;           // create a global variable named x
void valfun();  // function prototype (declaration)
int main() {
    int y;       // create a local variable named y
    x = 10;      // store a value into the global variable
    y = 20;      // store a value into the local variable
    cout << "From main(): x = " << x << endl;
    cout << "From main(): y = " << y << endl;
    valfun();    // call the function valfun
    cout << "\nFrom main() again: x = " << x << endl;
    cout << "From main() again: y = " << y << endl;
    return 0;
}
```

```
void valfun() {  
    int y;    // create a second local variable named y  
    y = 30;    // this only affects this local variable's value  
    cout << "\nFrom valfun(): x = " << x << endl;  
    cout << "\nFrom valfun(): y = " << y << endl;  
    x = 40;    // this changes x for both functions  
    return;  
}
```

- The output of the above program:

```
From main(): x = 10  
From main(): y = 20
```

```
From valfun(): x = 10  
From valfun(): y = 30
```

```
From main() again: x = 40  
From main() again: y = 20
```

Example 6.3.1 (cond.)

Content

1 Function declarations and definitions

2 Pass-by-value

3 Variable scope

4 Variable storage classes

5 Pass-by-reference

6 Passing arrays to functions

Variable storage class

- The lifetime of a variable is referred to as the **storage duration**, or **storage class**.
- Four available storage classes: **auto**, **static**, **extern** and **register**.
- If one of these class names is used, it must be placed before the variable's data type in a declaration statement.
- Examples:

```
auto int num;  
static int miles;  
register int dist;  
extern float price;  
extern float yld;
```

Local Variable Storage Classes

- **Local variables** can only be members of the `auto`, `static`, or `register` storage classes.
- **Automatic Variables:**
 - By default, local variables have the `automatic` storage class.
 - The term `auto` is short for automatic
 - **Automatic storage** duration refers to variables that exist only during the lifetime of the command block (such as a function) that contains them.

Example 6.4.1

```
#include <iostream>
using namespace std;
void testauto();           // function prototype
int main() {
    int count;              // count is a local auto variable
    for (count = 1; count <= 3; count++)
        testauto();
    return 0;
}
void testauto(){
    int num = 0;             // num is a local auto variable
    cout << "The value of the automatic variable num is "
         << num << endl;
    num++;
    return;
}
```

Example 6.4.1 (cond.)

- The output of the above program:

The value of the automatic variable num is 0

The value of the automatic variable num is 0

The value of the automatic variable num is 0

Static local variables

- In some applications, we want a function to **remember** values between function calls. This is the purpose of the **static** storage class.
- A **local static** variable is **not created and destroyed** each time the function declaring the static variable is called.
- Once created, local static variables *remain in existence* for the life of the program.

Example 6.4.2

```
#include <iostream>
using namespace std;
void teststatic();           // function prototype
int main() {
    int count;               // count is a local auto variable
    for(count = 1; count <= 3; count++)
        teststatic();
    return 0;
}

void teststatic() {
    static int num = 0;      // num is a local static variable
    cout << "The value of the static variable num is "
        << num << endl;
    num++;
    return;
}
```

Example 6.4.2 (cond.)

- The output of the above program:

The value of the static variable num is 0

The value of the static variable num is 1

The value of the static variable num is 2

Example 6.4.3

```
#include <iostream>
using namespace std;
int funct(int);           // function prototype
int main() {
    int count, value;      // count is a local auto variable
    for (count = 1; count <= 10; count++) {
        value = funct(count);
        cout << count << '\t' << value << endl;
    }
    return 0;
}

int funct (int x) {
    int sum = 100;         // sum is a local auto variable
    sum += x;
    return sum;
}
```

Note: The effect of increasing **sum** in **funct()**, before the function's return statement, is **lost** when control is returned to **main()**.

Example 6.4.3 (cond.)

- The output of the above program:

1	101
2	102
3	103
4	104
5	105
6	106
7	107
8	108
9	109
10	110

Example 6.4.4

```
#include <iostream>
using namespace std;
int funct(int);           // function prototype
int main() {
    int count, value;     // count is a local auto variable
    for (count = 1; count <= 10; count++) {
        value = funct(count);
        cout << count << '\t' << value << endl;
    }
    return 0;
}

int funct (int x) {
    static int sum = 100;  // sum is a local static variable
    sum += x;
    return sum;
}
```

Example 6.4.4 (cond.)

- The output of the above program:

1	101
2	103
3	106
4	110
5	115
6	121
7	128
8	136
9	145
10	155

Note:

- The **initialization** of **static** variables is done only **once** when the program is **first** compiled. At compile time, the variable is created and any initialization value is placed in it.
- All **static** variables are set to **zero** when **no explicit initialization** is given.

Register Variables

- Register variables have the same time duration as **automatic** variables.
- Register variables are **stored** in CPU's **internal registers** rather than in memory.
- Examples:

```
register int time;
```

```
register double difference;
```

Extern Variable

- Extern variable is a **Global** one which is declared in another file.

File header.h

```
1 #include <iostream>
2 using namespace std;
3 void increase_x();
4 void display_x();
```

File header.cpp

```
1 #include "header.h"
2 int x = 1;
3 void increase_x() {
4     x++;
5 }
6 void display_x() {
7     cout<<"Value of x: "
8         << x << endl;
9 }
```

File main.cpp

```
1 #include "header.h"
2 extern int x;
3 int main(){
4     increase_x();
5     display_x();
6     x++;
7     display_x();
8     cout<<"Value of x: "
9         << x << endl;
10    return 0;
11 }
```

- The output of the above program:
The value x is 2
The value x is 3
The value x is 3

Content

1 Function declarations and definitions

2 Pass-by-value

3 Variable scope

4 Variable storage classes

5 Pass-by-reference

6 Passing arrays to functions

Pass By Reference

- Two ways to invoke functions in many programming languages are:
 - call by value
 - call by reference
- When an argument is passed call **by value**, a **copy** of the argument's value is made and passed to the called function. Changes to the copy do **not affect** the original variable's value in the caller.
- With **call-by-reference**, the caller gives the called function the ability to access the caller's **data directly**, and to modify that data if the called function chooses so.
- To indicate that the function parameter is **passed-by-reference**, follow the parameter's type in the function prototype of function header by an ampersand (&).

Example 6.5.1

```
#include <iostream>
using namespace std;
int squareByValue (int);
void squareByReference (int & cRef);
int main() {
    int x = 2, z = 4;
    cout << "x = " << x << " before squareByValue\n"
         << "Value returned by squareByValue: "
         << squareByValue (x) << endl
         << "x = " << x << " after squareByValue\n" << endl;
    cout << "z = " << z << " before squareByReference" << endl;
    squareByReference (z);
    cout << "z = " << z << " after squareByReference" << endl;
    return 0;
}
```

The declaration `int & cRef` in the function header means “`cRef` is a reference parameter to an `int`”.

Example 6.5.1 (cond.)

```
int squareByValue (int a) {  
    return a *= a;    // caller's argument not modified  
}  
  
void squareByReference (int &cRef) {  
    cRef *= cRef;    // caller's argument modified  
}
```

- The output of the above program:
x = 2 before squareByValue
Value returned by squareByValue: 4
x = 2 after squareByValue

z = 4 before squareByReference
z = 16 after squareByReference

Content

1 Function declarations and definitions

2 Pass-by-value

3 Variable scope

4 Variable storage classes

5 Pass-by-reference

6 Passing arrays to functions

Passing Arrays To Functions

- To pass an array to a function, specify the name of the array **without any brackets**.

- For example, if array `hourlyTemperature` has been declared as

```
int hourlyTemperature[24];
```

- The following function call statement passes the array `hourlyTemperature` and its size to function `modifyArray`.

```
modifyArray (hourlyTemperature, size);
```

- For the function to receive an array through a function call, the function's parameter list must specify that an array will be received.
- For example, the function header for function `modifyArray` might be written as:

```
void modifyArray (int b[], int arraySize)
```

- Notice that the size of the array is **not required** between the array brackets.

Example 6.6.1

```
#include <iostream>
using namespace std;
int linearSearch (int [], int, int);
int main() {
    const int arraySize = 100;
    int a[arraySize], searchKey, element;
    for (int x = 0; x < arraySize; x++)    // create some data
        a[x] = 2 * x;
    cout << "Enter integer search key: " << endl;
    cin >> searchKey;
    element = linearSearch (a, searchKey, arraySize);
    if (element != -1)
        cout << "Found value in element " << element << endl;
    else
        cout << "Value not found" << endl;
    return 0;
}
```

Example 6.6.1 (cond.)

```
int linearSearch (int array[], int key, int sizeofArray)
{
    for (int n = 0; n < sizeofArray; n++)
        if (array[n] == key)
            return n;
    return -1;
}
```

Content

1 Function declarations and definitions

2 Pass-by-value

3 Variable scope

4 Variable storage classes

5 Pass-by-reference

6 Passing arrays to functions



VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Thank you!